Architectural Report on the Expansion of the Task Subsystem and Operational Governance in Stela

I. Constitutional Framework and Operational Ontology

The Stela platform operates not merely as a repository of code, but as a rigid cybernetic system governed by strict constitutional protocols. To understand the necessity and the mechanics of the proposed "Tasks Subsystem," one must first dissect the ontological foundations defined in the repository's constitution, specifically TRUTH.md and AGENTS.md. These documents do not serve as mere guidelines; they are the "physics" of the environment, establishing the immutable laws under which the Operator (Human), Integrator (Lead AI), and Contractor (Guest AI) interact.

The current operational phase, identified as "Phase Two" of the Projects Component expansion, represents a critical transition from ad-hoc capability (the Skills Subsystem) to formalized execution (the Tasks Subsystem). The analysis of the repository dump [1] indicates that while the system has successfully integrated a "Skills" layer (docs/library/skills/), the "Tasks" layer remains in a nascent, unhardened state, populated by "generic" artifacts (docs/library/tasks/B-TASK-XX.md) that violate the provenance-forward discipline of the platform.

1. The Role of the Task in the Agentic Hierarchy

Within the Stela ontology, a distinct separation of concerns exists between "Skills," "Tasks," and "Projects." Understanding this separation is a prerequisite for the architectural work proposed in DP-OPS-0016.

- **The Skill (S-LEARN-XX):** As defined in the recently established Skills Subsystem [1], a Skill represents *knowledge*. It is an atomic unit of capability—a specific verification loop, a security protocol, or a debugging heuristic. Skills are passive; they define *how* to do something but do not dictate *when* or *why* to do it. They are the tools in the belt.
- **The Project (P-PROJ-XX):** As hinted at in ops/bin/project and the future roadmap, a Project represents *payload*. It is the target material—the application code, the website, the infrastructure—upon which work is performed.
- **The Task (T-OPS-XX):** The Task represents *routine*. It is the execution logic that applies Skills to Projects to achieve a state change. A Task is a sequence of operations—an algorithm for labor—that orchestrates the Agent's behavior.

Currently, the repository relies on TASK.md to serve as the "Active Contract".[1] However, TASK.md functions primarily as a volatile work surface—a "RAM" for the current

thread—rather than a "Hard Drive" for standardized routines. This lack of a persisted library of standard tasks forces the Operator and Integrator to reinvent execution plans for common activities (e.g., Code Review, Verification, Planning), leading to "Drift" and inconsistency.

2. The Drift Anomaly and Generic Contamination

The forensic analysis of the repository dump reveals a significant structural anomaly in docs/library/tasks/. The presence of files named B-TASK-01.md through B-TASK-06.md [1] suggests the importation of "foreign tissue"—artifacts ripped from another repository without undergoing the requisite Stela hardening.

These generic tasks pose a "Drift" risk.[1] In the Stela lexicon, Drift occurs when the logic of the system deviates from the coordinates defined in TRUTH.md. Generic tasks typically lack:

1. **Stela-Specific Provenance:** They do not track their origin within the Stela SoP.md ledger.
2. **Jurisdictional Awareness:** They likely reference agents or roles (e.g., "Planner," "Architect") that may conflict with the strict Integrator/Contractor definitions in AGENTS.md.
3. **Hardened Stop Conditions:** Generic tasks rarely enforce the "Absolute Literalism" and "Anti-Drift Governance" required by AGENTS.md (e.g., stopping if a specific file path is missing).

Therefore, the primary objective of DP-OPS-0016 is not merely to "add" tasks, but to perform a "Host Rejection" of these foreign artifacts and replace them with native, constitution-compliant Operational Tasks (T-OPS).

3. The Governance of Interaction: Authority and Precision

The interaction between the Human Operator and the AI Agents is governed by "Staffing Protocols".[1] The Operator owns "final decisions, approvals, and secrets." The Integrator maintains "governance and structural integrity."

The expansion of the Tasks Subsystem serves the Integrator's mandate. by formalizing tasks, we reduce the cognitive load on the Contractor (Guest AI). Instead of relying on the Contractor to "hallucinate" a code review process, the System provides a rigid T-OPS specification. This aligns with the AGENTS.md requirement for "Linguistic Precision" and "Quantitative Reporting." A standardized task allows for verifying deviations (e.g., "step 3 omitted") rather than subjective assessments.

II. Architectural Evaluation: The Registry Pattern vs.

The Interface

A critical requirement of the user query is to "Evaluate if structuring TASK.md as the Registry is best practice." This requires a nuanced architectural debate regarding the Single Source of Truth (SSOT) and the separation of "Definition" from "Instance."

1. The Argument for TASK.md as Registry (The "Monolith" Approach)

One might argue that TASK.md, being the "Active Work Surface" [1], should also contain the definitions of all available tasks. This would theoretically centralize the workflow: the Agent opens TASK.md, sees the menu of all possible operations, and executes one.

However, this approach suffers from fatal flaws in the context of Large Language Model (LLM) operations:

- **Context Window Pollution:** If TASK.md contains the full text of 20 or 50 standardized tasks, every session initialization (./ops/bin/open) loads thousands of tokens of irrelevant data. The Agent needs to know *what to do now*, not *everything it could possibly do*.
- **Volatility Mismatch:** TASK.md is a "High Churn" file. It is edited minutes-by-minute as the Work Log is updated. A Registry is a "Low Churn" file—a standard library that changes only via governance events. Mixing high-frequency state with low-frequency definitions invites merge conflicts and historical noise.
- **The "Active Contract" Violation:** TASK.md is defined in TRUTH.md as the "active contract." If it becomes a library, it loses its specificity as a contract for *this* session.

2. The Argument for Separation (The "Interface" Approach)

The superior architectural pattern—and the one recommended by this report—is the **Interface/Implementation Separation**.

- **The Registry (docs/library/INDEX.md):** This file serves as the SSOT for *available* resources. It maps a stable identifier (e.g., task-ops-01) to a physical location (e.g., docs/library/tasks/T-OPS-01.md). This aligns with the existing pattern used for Skills.[1]
- **The Definition (docs/library/tasks/T-OPS-XX.md):** Each task lives in its own file. This ensures modularity, allows for independent versioning, and keeps the context clean (an Agent only loads the specific T-OPS file required for the job).
- **The Interface (TASK.md):** TASK.md acts as the *Entry Way* or *Loader*. It does not store the task definitions. Instead, it provides a mechanism (a standard header or instructions) to "import" a task into the active session.

Conclusion on Registry: Structuring TASK.md as the Registry is **NOT** best practice. It violates the principle of separation of concerns and degrades system performance (token efficiency). The correct pattern is to use TASK.md as the *Interface* that references the Registry in docs/library/INDEX.md.

3. The "Entry Way" Mechanism

The user query notes: "TASK.md WILL NEED TO BE MODIFIED SOMEHOW TO BE THE ENTRY WAY TO OTHER TASKS AS IS CURRENTLY THE RELATIONSHIP SAY SKILL.md.HAS WITH docs/library/skills/S-LEARN-XX FILES."

In the Skills Subsystem, SKILL.md acts as a "Factory" for creating skills and a "Log" for promoting them.[1] For the Tasks Subsystem, the relationship is slightly different. We are not "harvesting" tasks as frequently as skills; we are "invoking" them.

Therefore, TASK.md will be modified to include a reference pointer in **Section I (Required Context Load)** or **Section II (Scope)** that explicitly prompts the Operator/Integrator to check the Task Registry. Furthermore, the TASK.md template will be updated to allow the "Execution Plan" (Section III) to simply state "Execute T-OPS-XX" rather than writing out the steps manually. This transforms TASK.md into an orchestration manifest.

III. The Tasks Subsystem: Specification and Refactoring

This section details the specific architectural changes required to implement the Tasks Subsystem, replacing the generic B-TASK artifacts with Stela-native T-OPS artifacts.

1. Nomenclature and Taxonomy

To clearly distinguish between the "Foreign/Generic" legacy state and the "Native/Hardened" future state, we enforce a strict nomenclature shift:

- **Legacy:** B-TASK-XX (Base Task) — To be deprecated and deleted.
- **Target:** T-OPS-XX (Operational Task) — The new standard.

The T-OPS prefix signifies that these are *Operational* routines, distinct from *Learning* patterns (S-LEARN).

2. The T-OPS Standard Template

Every T-OPS file must adhere to a strict template derived from the AGENTS.md behavioral standards. This template ensures that an agent executing the task does not drift.

Required Sections for T-OPS:

1. **Provenance:** Origin (DP-ID), Jurisdiction (Role), and Revision History.
2. **Scope:** Strictly defined boundaries (e.g., "Production Payloads Only").
3. **Invocation Guidance:** The "Trigger" condition for using this task.
4. **Drift Preventers:** Hard stop conditions (e.g., "Stop if git status is dirty").

5. **Procedure:** The algorithmic steps, formatted as "Action -> Trap -> Solution."

3. Refactoring Plan: From Generic to Canon

We will perform a direct functional mapping and refactoring of the existing generic files [1] into the new T-OPS system.

A. Code Review Protocol (Refactoring B-TASK-01)

- **Current State:** Likely a generic checklist of "check for bugs."
- **New State (T-OPS-01):** A rigid **Code Review & Security Audit** protocol.
- **Stela Specifics:**
  - Must enforce the use of S-LEARN-05 (Security Protocols).[1]
  - Must require tools/lint_truth.sh to check for forbidden terminology.
  - Must require a check of docs/library/INDEX.md if documentation is touched.
  - **Jurisdiction:** Contractor (Code Reviewer).

B. Verification Loop (Refactoring B-TASK-02)

- **Current State:** A generic "run tests" command.
- **New State (T-OPS-02):** The **Verification Loop** protocol.
- **Stela Specifics:**
  - Must align with S-LEARN-01.[1]
  - Must enforce git status --porcelain (Clean State) before verification begins.
  - Must execute the tools/verify_tree.sh script to ensure repo hygiene.
  - Must produce a "Receipt" in the RESULTS artifact.

C. Orchestration (Refactoring B-TASK-03)

- **Current State:** Likely references "Planner" or "Architect" agents in a generic way.
- **New State (T-OPS-03): Agent Orchestration & Handoff**.
- **Stela Specifics:**
  - Defines the rigid handoff between **Integrator** (Strategy/Governance) and **Contractor** (Execution).
  - Specifies how to update TASK.md to reflect the handoff.
  - Enforces the "Anti-Drift Governance" from AGENTS.md.

D. Implementation Planning (Refactoring B-TASK-04)

- **Current State:** "Plan a feature."
- **New State (T-OPS-04): Implementation Planning & DP Drafting**.
- **Stela Specifics:**
  - The output of this task is **not** a conversational plan.
  - The output **MUST** be a valid DP-DRAFT formatted according to the TASK.md template.
  - This task essentially automates the "Architect Stance" defined in

docs/library/MANUAL.md.

E. Dead Code Cleanup (Refactoring B-TASK-05)

- **Current State:** Generic "delete unused files."
- **New State (T-OPS-05): Dead Code Cleanup & Hygiene**.
- **Stela Specifics:**
  - Must use tools/context_lint.sh to ensure no required context files are deleted.
  - Must update docs/library/INDEX.md if library files are removed.
  - Must verify against ops/lib/manifests/CONTEXT.md.

F. Documentation Sync (Refactoring B-TASK-06)

- **Current State:** "Update docs."
- **New State (T-OPS-06): Documentation Synchronization**.
- **Stela Specifics:**
  - Enforces the "Filing Doctrine" from TRUTH.md (Runs -> Ops, Explains -> Docs).
  - Validates that SoP.md is updated if Canon changes.
  - Runs tools/lint_library.sh to ensure the Registry is accurate.

IV. Hardening the TASK.md Interface

The Operator has identified specific failures in the current TASK.md template and its usage. These errors constitute "Process Drift" and must be rectified by hardening the template code itself.

1. The "Run OPEN" Loop Error

- **The Problem:** Current DPs instruct the worker to "Run OPEN" at the beginning of the task.
- **Analysis:** OPEN (ops/bin/open) is the **Session Initialization** mechanism. It generates the prompt that *loads* the agent. Asking an already-loaded agent to run OPEN is circular and redundant. It confuses the context window by reloading instructions that are already present.
- **The Fix:** The TASK.md template will be modified to remove any "Initialization" steps in the Execution Plan. Instead, a new section ## 0. FRESHNESS GATE is formalized (it exists in traces but needs hardening) which simply *verifies* that the session was opened correctly (e.g., "Confirm current branch is X"). It does not *run* the command.

2. The Artifact Copying Error

- **The Problem:** The Operator notes that DP writers are "adding artifacts for reference... from planner chat conversation."
- **Analysis:** This violates the "Single Source of Truth" principle. Copying text from a chat

into a DP creates a "forked" version of truth. If the original file changes, the DP copy is stale.
- **The Fix:** Stela enforces a **Pointer-First** constitution (AGENTS.md). The DP must **reference** files by path (e.g., "See docs/architecture/plan.md"), not duplicate their content. The TASK.md template will be updated with a warning comment: ``.

3. The Receipt Failure

- **The Problem:** "Closeout routine receipts not being produced for PORCELAIN DUMPS and OPEN."
- **Analysis:** Agents are lazy. If a step is not a rigid checkbox, they skip it. The "Definition of Done" in AGENTS.md requires equilibrium, which implies proof of state.
- **The Fix:** The TASK.md template Section E (Receipt) will be expanded into a **Mandatory Checklist**.
  - [ ] storage/handoff/DP-OPS-XX-RESULTS.md created
  - [ ] OPEN artifact preserved
  - [ ] Dump manifest preserved
    The agent will be instructed that failure to check these boxes constitutes a "Failed Task."

4. The Allowlist Ambiguity

- **The Problem:** "Task needs to be in allow list - it is written to every DP if I am not mistaken."
- **Analysis:** TASK.md is the living log. It is *always* written to. Requiring the Operator to manually add TASK.md to the "Target Files Allowlist" every time is friction. However, allowing it implicitly risks an agent editing the *template structure* rather than just the *log*.
- **The Fix:** We will keep TASK.md in the explicit allowlist requirements but add a pre-filled comment in the template: TASK.md (Work Log updates only). This reminds the agent that they have permission to write to the log, but *not* to refactor the template headings (unless that is the specific objective).

V. Future Roadmap: The Connective Tissue (Phase 3 & 4)

This Phase Two expansion is not an isolated event; it is the structural precursor to the full realization of the Stela Platform.

1. Phase Three: The Agents Subsystem

Following the formalization of Tasks, the next phase will expand the AGENTS.md file into a full directory subsystem (docs/library/agents/). Just as we are moving generic tasks to T-OPS, we

will eventually move the monolithic AGENTS.md definitions into specific persona files (e.g., A-CORE-01 Integrator, A-CORE-02 Contractor). This will allow for granular prompt engineering for each role, which can be invoked dynamically.

2. Phase Four: The Projects Component (ops/bin/project)

The ultimate goal is the ops/bin/project binary. This tool will tie Skills, Tasks, and Agents together.

- **Vision:** ops/bin/project start <project-id>
- **Mechanism:** The binary will read the Project's specific context, load the required S-LEARN skills, invoke the necessary A-CORE agent, and assign a T-OPS routine.
- **Dependency:** This automation is impossible without the Registry established in Phase Two. ops/bin/project cannot invoke "Code Review" if "Code Review" is just a vague concept. It needs to be able to call T-OPS-01.

Thus, the work in DP-OPS-0016 is the "Standardization of Parts" required before building the "Assembly Line."

VI. Implementation Plan (DP-OPS-0016)

The following section constitutes the executable Dispatch Packet. It allows the Contractor to perform the refactoring, registry establishment, and template hardening defined in the analysis above.

Execution Strategy

1. **Refactor:** The worker will delete the B-TASK files and create the T-OPS files. This is a destructive action regarding the old files, which is intentional (Host Rejection).
2. **Register:** The worker will manually update docs/library/INDEX.md to include the new T-OPS entries.
3. **Harden:** The worker will rewrite TASK.md to the new "Gold Standard."
4. **Verify:** The worker will run the full suite of linting tools to ensure the new registry entries resolve correctly.

Security and Safeguards

- **Isolation:** The Tasks Subsystem is isolated in docs/library/tasks/. It does not touch ops/lib/manifests/CONTEXT.md directly, avoiding the "Context Hazard" (bloating the context). The Registry (INDEX.md) acts as the buffer.
- **Context Hazard:** We must ensure that T-OPS files are **not** added to the Context Manifest. They are strictly "Load on Demand."
- **Recovery:** If the refactor fails, SoP.md provides the rollback point.

The Dispatch Packet

DP-OPS-0016: EXPANSION OF TASK SUBSYSTEM AND PROJECT PHASE II

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: work/tasks-sub-system

Required Work Branch: work/tasks-sub-system

Base HEAD: 7e890ab96

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- docs/library/INDEX.md (Registry)
- TASK.md (Template to be modified)
- docs/library/tasks/ (Directory to be refactored)

II) SCOPE & SAFETY

- **Objective:** Establish the Tasks Subsystem by refactoring generic tasks into Stela Canon (T-OPS), establishing the Registry pattern in INDEX.md, and hardening the TASK.md template.

- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin/project (Phase Three).
  - No changes to existing Skills (S-LEARN).
  - Do not edit ops/lib/manifests/CONTEXT.md.

Target Files allowlist (hard gate)

- TASK.md
- SoP.md
- docs/library/INDEX.md
- docs/library/tasks/T-OPS-01.md (new)
- docs/library/tasks/T-OPS-02.md (new)
- docs/library/tasks/T-OPS-03.md (new)
- docs/library/tasks/T-OPS-04.md (new)
- docs/library/tasks/T-OPS-05.md (new)
- docs/library/tasks/T-OPS-06.md (new)
- docs/library/tasks/B-TASK-01.md (delete)
- docs/library/tasks/B-TASK-02.md (delete)
- docs/library/tasks/B-TASK-03.md (delete)
- docs/library/tasks/B-TASK-04.md (delete)
- docs/library/tasks/B-TASK-05.md (delete)
- docs/library/tasks/B-TASK-06.md (delete)
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**.

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The repository contains generic tasks (B-TASK-XX) imported from an external source. These violate Stela's provenance and drift-prevention standards. TASK.md requires structural hardening to prevent DP composition errors (OPEN loops, artifact copying).
- **Drift:** 6 unverified task files; TASK.md template permits "Run OPEN" loops; Receipt bundles are inconsistent; TASK.md not formally established as Interface to Registry.
- **Desired State:** docs/library/tasks/ contains strict T-OPS artifacts. TASK.md is the hardened interface pointing to the Registry. The Registry (INDEX.md) is the SSOT for available routines.

B) REQUEST

1. Refactor Task Library (The Purge & Replace):

- **Delete:** docs/library/tasks/B-TASK-01.md through B-TASK-06.md.
- **Create:** docs/library/tasks/T-OPS-01.md (Code Review Protocol).
  - Use AGENTS.md linguistic standards (no contractions).
  - Include Provenance, Scope, Invocation, Drift Preventers, Procedure.
  - Reference S-LEARN-05 (Security).
- **Create:** docs/library/tasks/T-OPS-02.md (Verification Loop).
  - Enforce git status --porcelain check.
  - Align with S-LEARN-01.
- **Create:** docs/library/tasks/T-OPS-03.md (Agent Orchestration).
  - Define handoff between Integrator and Contractor.
- **Create:** docs/library/tasks/T-OPS-04.md (Implementation Planning).
  - Output must be DP-DRAFT, not text.
- **Create:** docs/library/tasks/T-OPS-05.md (Dead Code Cleanup).
  - Require verify_tree.sh check.
- **Create:** docs/library/tasks/T-OPS-06.md (Documentation Sync).
  - Update docs/library/INDEX.md and SoP.md.

2. Establish Registry:

- Update docs/library/INDEX.md:
  - Remove B-TASK entries.
  - Add a new section: ## --- 6. Standard Tasks (Execution Routines) ---.
  - Register all new T-OPS files.

3. Harden TASK.md Template:

- **Remove** instructions to run OPEN or dump from the "Receipt" section instructions (these are pre-flight).
- **Add** strict "Receipt Bundle Checklist" to Section E (as seen in SoP hardening).
- **Add** "Task Registry Pointer" to Section 1 (Active Context), instructing the Operator to reference docs/library/INDEX.md for available routines.
  - Text: See docs/library/INDEX.md for Standard Tasks (T-OPS).
- **Fix** the "Authoring-Time Artifacts" issue by adding comment: ``.
- **Add** TASK.md (Work Log updates only) to the default Target Files Allowlist example.

C) CHANGELOG

- **Deleted:** docs/library/tasks/B-TASK-[01-06].md.
- **Added:** docs/library/tasks/T-OPS-[01-06].md.
- **Modified:** docs/library/INDEX.md (Registry update).
- **Modified:** TASK.md (Template hardening).
- **Modified:** SoP.md (History).

D) PATCH / DIFF

- Format: Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- bash tools/context_lint.sh
- bash tools/lint_truth.sh
- bash tools/lint_library.sh (Crucial: verifies the new T-OPS files are registered).
- bash tools/verify_tree.sh

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/skill/skill_lib.sh harvest if a new reusable pattern emerges.
- Proof required in RESULTS.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0016-RESULTS.md (required)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

VII. Detailed Analysis of the Generic Task Refactoring (T-OPS Specification)

To ensure the Contractor generates valid artifacts during the execution of DP-OPS-0016, this section provides the detailed "Genome" for the new T-OPS files. This analysis is derived from the "Best Practices" implicit in the AGENTS.md and S-LEARN files.

1. T-OPS-01: Code Review Protocol

The generic B-TASK-01 likely contained a loose checklist. The new T-OPS-01 must be a rigorous security gate.

- **Drift Preventer:** "Stop if the diff includes changes to main branch configuration or

.github workflows without explicit authorization."
- **Procedure Step 1:** "Secrets Scan." The agent must grep for standard key patterns (sk-, ghp_, Bearer).
- **Procedure Step 2:** "Canon Compliance." The agent must verify that no new markdown files are created outside of the docs/ or projects/ trees (as per verify_tree.sh).

2. T-OPS-02: Verification Loop

The generic B-TASK-02 was a simple "Test" command. T-OPS-02 elevates this to a System Integrity Check.

- **Invocation:** "Use this task before marking any DP as complete."
- **Drift Preventer:** "Stop if git status --porcelain is not empty. Tests must run on clean state."
- **Integration:** Explicitly calls tools/context_lint.sh and tools/lint_truth.sh. This ties the Task directly to the Tools subsystem, reinforcing the "Stela Stack."

3. T-OPS-03: Agent Orchestration

This task replaces the vague B-TASK-03 with a definition of the Stela "Staffing Protocol."

- **Concept:** It defines how an Integrator (Lead) passes a task to a Contractor (Guest).
- **Procedure:**
  1. Integrator defines Scope in TASK.md.
  2. Contractor accepts Scope.
  3. Contractor executes.
  4. Contractor updates Work Log.
  5. Integrator reviews Receipt.

4. T-OPS-04: Implementation Planning

This is critical for the "Architect" persona. B-TASK-04 likely allowed for conversational planning. T-OPS-04 restricts the output.

- **Constraint:** "The output of this task is a file in storage/dp/intake/DP-DRAFT-XX.md."
- **Logic:** It forces the planner to think in terms of "Target Files Allowlist" and "Verification Steps" *before* any code is written. This is the implementation of "Governance-as-Code" at the planning stage.

VIII. Conclusion and Continuity

The execution of DP-OPS-0016 marks the maturity of the Stela Platform's "Runtime." By purging the generic artifacts and establishing a strict Tasks Subsystem, we achieve three critical outcomes:

1. **Provenance:** Every executable routine in the library now has a known origin and adheres

to the Constitution.
2. **Stability:** The TASK.md template is hardened against common failure modes (OPEN loops, missing receipts), reducing "Process Drift."
3. **Scalability:** The establishment of the Task Registry in INDEX.md paves the way for the automated orchestration of the future ops/bin/project tool.

The system is now prepared to transition from "Manual Governance" to "Assisted Governance," where the tools themselves enforce the rules of the road.

NEXT: Execute DP-OPS-0016.

FOLLOWING: Phase Three - Expansion of Agents Subsystem.

Works cited

1. dump-full-work-tasks-sub-system-7e890ab96.txt